Double Hashing

Last Updated : 29 Mar, 2024

Double hashing is a collision resolution technique used in hash tables. It works by using two hash functions to compute two different hash values for a given key. The first hash function is used to compute the initial hash value, and the second hash function is used to compute the step size for the probing sequence.

Double hashing has the ability to have a low collision rate, as it uses two hash functions to compute the hash value and the step size. This means that the probability of a collision occurring is lower than in other collision resolution techniques such as linear probing or quadratic probing.

However, double hashing has a few drawbacks. First, it requires the use of two hash functions, which can increase the computational complexity of the insertion and search operations. Second, it requires a good choice of hash functions to achieve good performance. If the hash functions are not well-designed, the collision rate may still be high.

Advantages of Double hashing

- The advantage of Double hashing is that it is one of the best forms of probing, producing a uniform distribution of records throughout a hash table.
- This technique does not yield any clusters.
- It is one of the effective methods for resolving collisions.

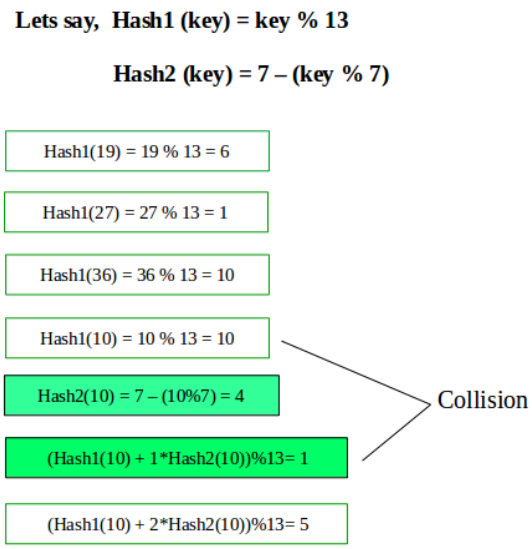
Double hashing can be done using :
(hash1(key) + i * hash2(key)) % TABLE_SIZE
Here hash1() and hash2() are hash functions and TABLE_SIZE is size of hash table.
(We repeat by increasing i when collision occurs)

Method 1: First hash function is typically hash1(key) = key % TABLE_SIZE
A popular second hash function is **hash2(key) = PRIME - (key % PRIME)** where

PRIME is a prime smaller than the TABLE_SIZE.

A good second Hash function is:

- It must never evaluate to zero
- Just make sure that all cells can be probed



Below is the implementation of the above approach:

CPP Java Python3 C# JavaScript

```cpp
/*
** Handling of collision via open addressing
** Method for Probing: Double Hashing
*/

#include <iostream>
#include <vector>
#include <bitset>
using namespace std;
#define MAX_SIZE 10000001ll

class doubleHash {

    int TABLE_SIZE, keysPresent, PRIME;
    vector<int> hashTable;
    bitset<MAX_SIZE> isPrime;

    /* Function to set sieve of Eratosthenes. */
```

```
    void __setSieve(){
        isPrime[0] = isPrime[1] = 1;
        for(long long i = 2; i*i <= MAX_SIZE; i++)
            if(isPrime[i] == 0)
                for(long long j = i*i; j <= MAX_SIZE; j += i)
                    isPrime[j] = 1;

    }

    int inline hash1(int value){
        return value%TABLE_SIZE;
    }

    int inline hash2(int value){
        return PRIME - (value%PRIME);
    }

    bool inline isFull(){
        return (TABLE_SIZE == keysPresent);
    }

    public:

    doubleHash(int n){
        __setSieve();
        TABLE_SIZE = n;

        /* Find the largest prime number smaller than hash table's
size. */
        PRIME = TABLE_SIZE - 1;
        while(isPrime[PRIME] == 1)
            PRIME--;

        keysPresent = 0;

        /* Fill the hash table with -1 (empty entries). */
        for(int i = 0; i < TABLE_SIZE; i++)
            hashTable.push_back(-1);
    }

    void __printPrime(long long n){
        for(long long i = 0; i <= n; i++)
```

```
            if(isPrime[i] == 0)
                cout<<i<<", ";
        cout<<endl;
    }

    /* Function to insert value in hash table */
    void insert(int value){

        if(value == -1 || value == -2){
            cout<<("ERROR : -1 and -2 can't be inserted in the
table\n");
        }

        if(isFull()){
            cout<<("ERROR : Hash Table Full\n");
            return;
        }

        int probe = hash1(value), offset = hash2(value); // in
linear probing offset = 1;

        while(hashTable[probe] != -1){
            if(-2 == hashTable[probe])
                break;                                  // insert
at deleted element's location
            probe = (probe+offset) % TABLE_SIZE;
        }

        hashTable[probe] = value;
        keysPresent += 1;
    }

    void erase(int value){
        /* Return if element is not present */
        if(!search(value))
            return;

        int probe = hash1(value), offset = hash2(value);

        while(hashTable[probe] != -1)
            if(hashTable[probe] == value){
```

```
                hashTable[probe] = -2;          // mark element as
deleted (rather than unvisited(-1)).
                keysPresent--;
                return;
            }
            else
                probe = (probe + offset) % TABLE_SIZE;

    }

    bool search(int value){
        int probe = hash1(value), offset = hash2(value), initialPos
= probe;
        bool firstItr = true;

        while(1){
            if(hashTable[probe] == -1)                  // Stop
search if -1 is encountered.
                break;
            else if(hashTable[probe] == value)          // Stop
search after finding the element.
                return true;
            else if(probe == initialPos && !firstItr)   // Stop
search if one complete traversal of hash table is completed.
                return false;
            else
                probe = ((probe + offset) % TABLE_SIZE);  // if
none of the above cases occur then update the index and check at
it.

            firstItr = false;
        }
        return false;
    }

    /* Function to display the hash table. */
    void print(){
        for(int i = 0; i < TABLE_SIZE; i++)
            cout<<hashTable[i]<<", ";
        cout<<"\n";
    }

};
```

```
int main(){
    doubleHash myHash(13); // creates an empty hash table of size
13

    /* Inserts random element in the hash table */

    int insertions[] = {115, 12, 87, 66, 123},
        n1 = sizeof(insertions)/sizeof(insertions[0]);

    for(int i = 0; i < n1; i++)
        myHash.insert(insertions[i]);

    cout<< "Status of hash table after initial insertions : ";
myHash.print();



    /*
    ** Searches for random element in the hash table,
    ** and prints them if found.
    */

    int queries[] = {1, 12, 2, 3, 69, 88, 115},
        n2 = sizeof(queries)/sizeof(queries[0]);

    cout<<"\n"<<"Search operation after insertion : \n";

    for(int i = 0; i < n2; i++)
        if(myHash.search(queries[i]))
            cout<<queries[i]<<" present\n";


    /* Deletes random element from the hash table. */

    int deletions[] = {123, 87, 66},
        n3 = sizeof(deletions)/sizeof(deletions[0]);

    for(int i = 0; i < n3; i++)
        myHash.erase(deletions[i]);

    cout<< "Status of hash table after deleting elements : ";
myHash.print();
```

```
    return 0;
}
```

Output

```
Status of hash table after initial insertions : -1, 66, -1, -1, -1,
-1, 123, -1, -1, 87, -1, 115, 12,

Search operation after insertion :
12 present
115 present
Status of hash table after deleting elements : -1, -2, -1, -1, -1,
-1, -2, -1, -1, -2, -1, 115, 12,
```

Time Complexity:

- **Insertion:** O(n)
- **Search:** O(n)
- **Deletion:** O(n)

Auxiliary Space: O(size of the hash table).

Next Article
Load Factor and Rehashing

Similar Reads

Hashing in Data Structure

Hashing is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access. Hashing involves mapping data to a specific index in a hash table (an array of...

3 min read

Introduction to Hashing

Hashing refers to the process of generating a small sized output (that can be used as index in a table) from an input of typically large and variable size. Hashing uses mathematical formulas known as hash...

7 min read

What is Hashing?

Hashing refers to the process of generating a fixed-size output from an input of variable size using the mathematical formulas known as hash functions. This technique determines an index or location for the...

3 min read

Index Mapping (or Trivial Hashing) with negatives allowed

Index Mapping (also known as Trivial Hashing) is a simple form of hashing where the data is directly mapped to an index in a hash table. The hash function used in this method is typically the identity...

7 min read

Separate Chaining Collision Handling Technique in Hashing

Separate Chaining is a collision handling technique. Separate chaining is one of the most popular and commonly used techniques in order to handle collisions. In this article, we will discuss about what is...

3 min read

Open Addressing Collision Handling technique in Hashing

Open Addressing is a method for handling collisions. In Open Addressing, all elements are stored in the hash table itself. So at any point, the size of the table must be greater than or equal to the total number...

7 min read

Double Hashing

Double hashing is a collision resolution technique used in hash tables. It works by using two hash functions to compute two different hash values for a given key. The first hash function is used to comput...

15+ min read

Load Factor and Rehashing

Prerequisites: Hashing Introduction and Collision handling by separate chaining How hashing works: For insertion of a key(K) - value(V) pair into a hash map, 2 steps are required: K is converted into a small...

15+ min read

Easy problems on Hashing

Intermediate problems on Hashing